

\$ KUBECTL APPLY

Kubernetes 101

El arranque del libro

Antes de empezar · Arquitectura de Kubernetes

Apéndice D: El clúster por dentro

Treinta y una páginas y los ocho diagramas, tal y como aparecen en el libro.

REGALO PARA SUSCRIPTORES

Javier Vela Aylón

www.javivela.dev · [@jvela](https://twitter.com/jvela)

Antes de empezar

Tienes una aplicación web. Se ejecuta en un servidor. Funciona.

Un martes a las tres de la mañana el proceso muere. Nadie se entera hasta las nueve, cuando llama un cliente. Entrás por SSH, lo arrancas otra vez y sigues con lo tuyo pensando que habría que montar algo que lo levante solo.

Montas ese algo: un script que cada minuto mira si el proceso está vivo y, si no lo está, lo arranca. Ya no es tu problema a las tres de la mañana. Acabas de escribir, sin saberlo, la idea central de Kubernetes: tú declaras cómo quieres que estén las cosas, y algo se ocupa de conseguirlo. No das pasos, das un resultado. Lo que decide qué hacer para llegar hasta ahí es el sistema, y no para nunca de comprobarlo.

Pero un servidor sigue siendo un servidor. El día que se cae la máquina entera, tu script se cae con ella. Necesitas la aplicación en tres máquinas. Y en cuanto tienes tres copias aparece una pregunta nueva: ¿a cuál llaman los demás? No puedes repartir tres direcciones IP por ahí, porque esas copias van a morir y a nacer constantemente —esa es justo la gracia— y ninguna dirección va a durar. Hace falta un nombre fijo que apunte siempre a las que puedan atender las peticiones.

Y hay una tercera cosa. Lo de las copias intercambiables está muy bien para la web, pero tu base de datos no es intercambiable: tiene un disco que no puede perder y un nombre que no puede cambiar. Da igual cuántas copias levantes, no son la misma cosa que las anteriores.

Kubernetes es ese script, hecho bien, para un montón de máquinas a la vez. Y todo él se apoya en tres ideas:

- **Declaras el resultado, no los pasos.** Escribes que quieres tres copias de algo y quién puede hablar con quién. Kubernetes compara constantemente lo que hay con lo que pediste, y si no coincide, actúa. No hay un botón de “desplegar”: hay un estado deseado y algo que lo persigue.
- **Lo que se ejecuta es un Pod, y un Pod es de usar y tirar.** Es la unidad más pequeña que Kubernetes sabe arrancar: uno o varios contenedores que comparten red y disco, y que nacen y mueren juntos. Un Pod no se repara, se sustituye; que muera no es una avería, es el funcionamiento normal.

Ahí encaja el nombre fijo que buscabas hace dos párrafos, y también quien reponga los Pods que falten: las dos piezas existen porque el Pod no dura.

- **Pero no todo es sustituible.** Dos copias de tu base de datos no son lo mismo que dos copias de la web, por el motivo que acabas de leer. Saber qué pieza es de usar y tirar y cuál no está detrás de la mitad de las decisiones de este libro.

Con esas tres ideas ya puedes leer el mapa. Los nombres que vas a ver a partir de aquí —Deployment, StatefulSet, Service, DaemonSet— son los objetos con los que Kubernetes construye todo esto. Cada uno tiene su capítulo; de momento basta con saber a qué pieza pertenece cada nombre.

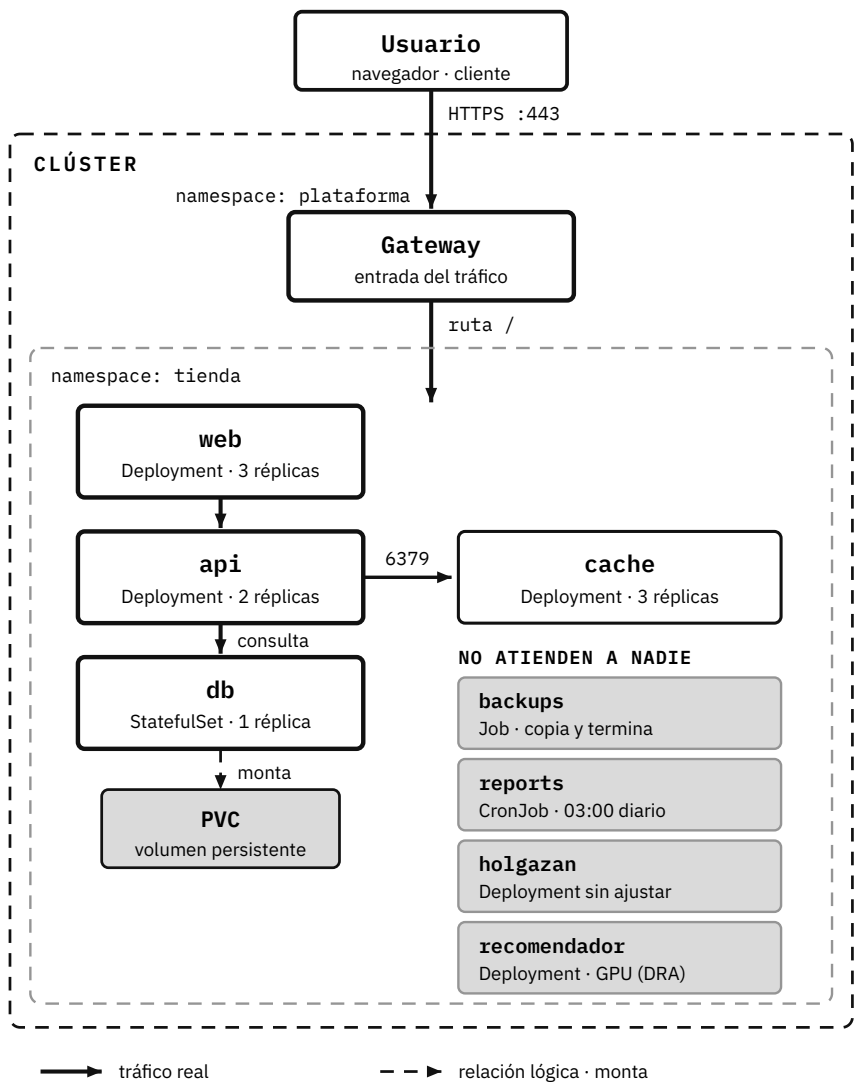


Figura 1: La aplicación tienda: Namespaces y piezas

Sigue el trazo grueso y tendrás el camino de una petición: Usuario, Gateway, web, api y db. Abajo, en su banda, las cuatro piezas que están en el sistema pero fuera de ese camino. Falta el agent, que va en la figura siguiente.

Las piezas

El clúster tiene tres nodos y tres Namespaces: tienda para la aplicación, plataforma para lo que deja entrar tráfico o toca el nodo, y observabilidad para Prometheus, Grafana y Loki. Dentro viven diez piezas, y cada una está aquí porque enseña algo que las demás no pueden enseñar. Se leen como las dibuja el mapa: primero las que atraviesa una petición, después las que están al lado, y por último las que no atienden a nadie.

Las cuatro del camino:

gateway — *la que deja pasar el tráfico de fuera*. La puerta de entrada, y donde termina el TLS¹. Vive en `plataforma`, y a propósito: quien abre la puerta no debería estar dentro de la casa.

web — *la que se puede tirar sin consecuencias*. El frontend, tres copias idénticas. A cualquiera de ellas le da exactamente igual desaparecer, y esa indiferencia es justo lo que permite cambiarle la versión sin cortar el servicio. Por eso es un Deployment.

api — *la que crece sola*. El backend, y la pieza que sufre los picos de tráfico. Es un Deployment como `web`, con una diferencia: cuántas copias hay no lo escribes tú, lo decide un autoescalador según la gente que llegue.

db — *la que no puede perder su disco*. PostgreSQL, con una sola copia y con un disco y un nombre que son suyos y de nadie más. Por eso no es un Deployment, es un StatefulSet. Si entiendes por qué esta pieza es distinta de las dos anteriores, entiendes la mitad de Kubernetes.

Las dos que están al lado del camino:

cache — *la que obliga a decidir en qué nodo cae cada copia*. Redis, tres copias, y hay tres nodos. La pregunta parece tonta hasta que la haces: ¿quién garantiza que caiga una en cada uno? Nadie, hasta que lo declaras.

agent — *la que va en todas las máquinas*. Recoge los logs, y hay una instancia por nodo en lugar de una por aplicación: su número tampoco lo pones tú, lo pone el clúster al contar sus máquinas. Necesita leer el disco del nodo, y esa necesidad la destierra de tienda a `plataforma`.

¹Transport Layer Security. Protocolo criptográfico que cifra los datos intercambiados entre dos aplicaciones a través de una red, como Internet.

Y las cuatro que no atienden a nadie. Están en el mapa, en la banda de abajo, y aparecen cuando un capítulo las necesita:

- **backups** es un Job: copia la db y termina. Empezar y terminar es justo lo que distingue una tarea de un servicio.
- **reports** es el informe de ventas, y es un CronJob: la misma idea, pero cada noche a las tres y sin que nadie se lo pida.
- **holgazan** pide dos CPU y cuatro gigas de memoria para usar una décima parte. Existe para que el VPA² tenga algo que arreglar, y para que veas cuánto se despilfarra cuando nadie mira los números.
- **recomendador** necesita una GPU. Es el único que pide hardware en vez de solo CPU y memoria, y en el clúster solo hay una: por eso es con lo que se explica DRA³.

Los nombres exactos de cada objeto, sus imágenes y sus puertos están en el Apéndice B.

²VerticalPodAutoscaler. Ajusta automáticamente las `requests` y `limits` de CPU y memoria de un Pod según su consumo real.

³Dynamic Resource Allocation. Mecanismo para solicitar dispositivos especializados (GPUs, FPGAs, NICs...) con requisitos concretos (modelo, memoria, topología). El usuario describe lo que necesita en un `ResourceClaim` y el `kube-scheduler` elige nodo y dispositivo a la vez.

LOS 3 NODOS DEL CLÚSTER

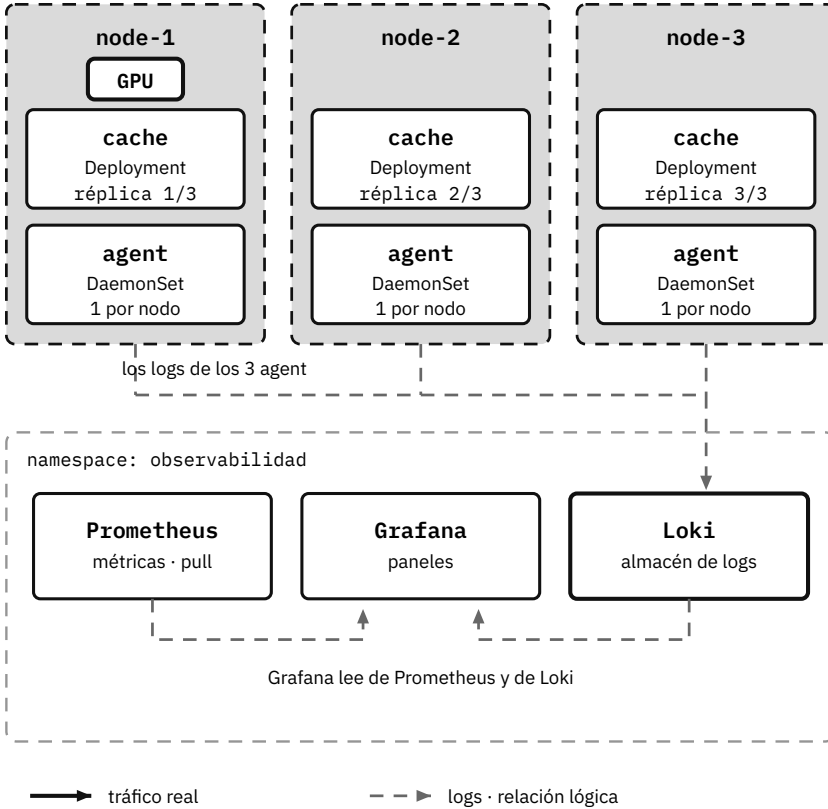


Figura 2: La aplicación tienda: los tres nodos y la observabilidad

Una réplica de cache y un agent en cada nodo, y la GPU solo en node-1. Esa escasez es la razón de ser de DRA: el recomendador no pide una GPU como quien pide CPU, la reclama, y si otro Pod la quisiera a la vez, uno de los dos esperaría.

Cómo se construye

Este es el recorrido del sistema, capítulo a capítulo. No es el índice completo — *Introducción a los contenedores* y *Arquitectura* son de conceptos y no añaden piezas —, pero sí está todo lo que construye la tienda. Si en algún momento te pierdes, vuelve aquí:

Capítulo	Qué añade al sistema
Hablar con el clúster	El Namespace tienda, y el primer Pod web, que creas y borras
Objetos de Kubernetes	web vuelve, todavía como un Pod suelto
Workloads	web pasa a Deployment · llega db (StatefulSet) · agent (DaemonSet) · backups y reports
Escalado	api aprende a escalar sola (HPA) · aparece el ho1gazan (VPA)
Red	Los Services de web, api y cache · el gateway · la NetworkPolicy que aísla la db
Configuración	web-config y el Secret db-credentials
Almacenamiento	El disco de la db: pv-db y datos-db-0
Seguridad	agent-sa y api-sa, RBAC y hardening ⁴ de api
Policías	El presupuesto del Namespace tienda
Scheduling	El Deployment de cache, una réplica por nodo · el PDB de web · el recomendador y su GPU
Trabajando con el clúster	Operar todo lo anterior
Extensibilidad	Ninguna pieza nueva: el tipo Promocion y el operador que lo atiende
Observabilidad	El Namespace observabilidad

El sistema, en YAML y listo para aplicar, está en el repositorio del libro: <https://github.com/fjvela/kubernetes-101>. Están las diez piezas, aunque

⁴Hardening. Conjunto de medidas que reducen la superficie de ataque de un sistema: quitar los privilegios que no se usan, cerrar lo que no hace falta y dejar solo lo mínimo para que funcione. En un Pod se concreta en el `securityContext` y en los perfiles de Pod Security Admission. La referencia habitual es la *Kubernetes Hardening Guide* de la NSA y CISA, en la bibliografía.

el recomendador va aparte: sin una GPU se aplica igual, pero se queda esperando. El Apéndice B te dice qué hay allí y cómo desplegarlo.

La otra mitad de este libro

Este libro tiene una segunda mitad que no está en el papel, y es gratis.

Cada capítulo tiene su laboratorio en un clúster de Kubernetes real que arranca en tu navegador: no hay nada que instalar, ni Docker, ni un clúster que montar. Son quince módulos y cincuenta y cinco laboratorios sobre el mismo sistema que se construye en estas páginas, con la misma tienda y los mismos nombres. Cada tarea se verifica sola, así que sabes si lo has hecho bien sin que nadie te lo diga.

No son ejercicios de repetir lo leído. Siete son **challenges**: te dan un clúster y un problema, sin guion. La mayoría te entrega un sistema que no funciona y te pide arreglarlo, que es como se aprende esto de verdad.

Y hay un módulo que no corresponde a ningún capítulo, porque no hay nada nuevo que leer en él: el **reto final**, donde apagas el kube-scheduler de un clúster vivo para comprobar qué se cae y qué sigue funcionando, que no es lo que casi nadie espera. En otros tres sitios el laboratorio llega más lejos que estas páginas: en *Contenedores* construyes tú la imagen de la tienda capa a capa; el operador del capítulo de Extensibilidad, que aquí se explica, allí se escribe entero en Go; y en *Scheduling* llegas a publicar a mano el dispositivo que aquí solo se reclama, para ver al selector descartarlo.

<https://www.javivela.dev/libros/kubernetes-101/practica>

No hay código, ni cupón, ni nada que validar: si has llegado hasta aquí, es tuyo. Empieza por el módulo *Fundamentos*, donde le sacas a `kubectl` la petición HTTP que manda al clúster y después creas el primer Pod de la tienda, lo rompes y lo diagnosticas.

Capítulo	Módulo del laboratorio
Introducción a los contenedores	Contenedores – construyes tienda/api
Arquitectura de Kubernetes · Apéndice D	El clúster por dentro – sobre kubeadm
Hablar con el clúster	Fundamentos – <code>kubectl</code> por dentro, tu primer Pod y tu primer Pod roto
Objetos de Kubernetes	Fundamentos – labels, selectors y annotations

Capítulo	Módulo del laboratorio
Workloads	Workloads — el módulo más largo, con ocho lecciones y dos challenges
Escalado	Escalado — HPA con carga real y VPA sobre el holgazán
Red	Red — Services, Ingress, Gateway API, NetworkPolicy y CoreDNS
Configuración	Configuración — y el reto del registro privado
Almacenamiento	Almacenamiento — del emptyDir al disco que sobrevive
Seguridad	Seguridad — RBAC y Pod Security Admission
Policies	Policies — cuotas y LimitRanges
Scheduling	Scheduling — dos Pods en Pending, un drenado bloqueado y la GPU que no existe
Trabajando con el clúster	Trabajando con el clúster — depuración en formato misión
Extensibilidad	Extensibilidad — el operador entero, en Go
Observabilidad	Observabilidad — Prometheus, Grafana, Loki y VictoriaMetrics
—	Reto final — apaga el scheduler y diagnostica

En cada capítulo encontrarás también, en su recuadro, qué se hace en el módulo que le corresponde.

Condiciones de la parte práctica

Los laboratorios están alojados en una plataforma externa. El autor no participa en ella ni en su gestión, y el funcionamiento, la disponibilidad y el tratamiento de datos son responsabilidad suya, no del autor. El enlace de alta que encontrarás en la página de la parte práctica es un enlace de referido: si te registras a través de él,

el autor puede recibir una contrapartida de la plataforma. No cambia nada para ti, porque la parte práctica es gratuita y no requiere ninguna compra.

Se ofrecen tal cual, sin garantía de funcionamiento futuro. Los cambios que introduzcan Kubernetes u otras herramientas (breaking changes⁵) pueden afectar a los escenarios. El autor se compromete a hacer todo lo posible por mantenerlos actualizados y al día, pero no puede garantizar su disponibilidad ni su funcionamiento permanente. Si encuentras algo roto, avisa a libros@javivela.dev es la mejor forma de ayudar.

Cuatro diferencias entre estas páginas y el laboratorio

El libro explica el caso general; el laboratorio tiene que arrancar de verdad sobre un clúster concreto, y eso obliga a unas cuantas concesiones. Ninguna cambia lo que se aprende, pero conviene saberlas para no pensar que has copiado mal:

- **Los nombres de los nodos.** Aquí los nodos se llaman `node-1`, `node-2` y `node-3`. En el laboratorio los pone la plataforma al registrarlos, con el cero delante —`node-01` y `node-02`— y con dos nodos programables en lugar de tres. Si copias un `kubectl describe node node-1` o un `nodeName: node-3` tal cual, te responderá `NotFound`: mira el nombre real con `kubectl get nodes`. Y donde aquí cache reparte tres réplicas entre tres nodos, allí verás dos entre dos. La regla que aprendes es la misma.
- **Las imágenes.** `tienda/web:1.4.2` y `tienda/api:2.1.0` son nombres ilustrativos: no existen en ningún registro público. El laboratorio usa imágenes reales espejadas en el registro de la plataforma. La excepción es el módulo de *Contenedores*, donde construyes tú la `tienda/api:2.1.0` y el nombre pasa a ser literal.
- **Los puertos.** Esas imágenes reales son un `nginx` y escuchan en el **80**, no en el 8080 de estas páginas, así que los *Services* y las *probes* del laboratorio apuntan al 80. La *api* que construyes en *Contenedores* es la excepción: esa sí escucha en el 8080.
- **El recomendador no llega a arrancar en ningún sitio.** Es la única pieza de la tienda que pide una GPU, y ni el laboratorio ni tu portátil tienen ninguna. Aplicarlo sí se puede, y de hecho su laboratorio va más lejos que estas páginas: llega a publicar un dispositivo de mentira a mano para ver al selector elegir. El Pod se queda a un paso, en el driver.

⁵Breaking change. Cambio en una nueva versión que rompe la compatibilidad con la anterior, obligando a adaptar manifiestos o herramientas antes de actualizar.

Versión de referencia

Todo lo descrito en este libro corresponde a Kubernetes 1.37 (agosto de 2026), salvo que se indique lo contrario.

Cuando el texto no dice nada de versiones, es porque en un clúster actual la cosa sencillamente funciona. El **Apéndice C** recoge, capítulo a capítulo, desde cuándo funciona cada pieza y en qué estado está, y es el sitio al que ir si tu clúster es más antiguo que la versión de referencia.

El cuerpo del libro solo menciona una versión en los dos casos en que te obliga a hacer algo: cuando una funcionalidad está en **alpha o beta** y hay que activarla, y cuando algo está **deprecado** y hay que migrar.

Erratas y actualizaciones. Las correcciones posteriores a la impresión y los cambios de versión que afecten a lo que aquí se cuenta se publican en <https://www.javivela.dev/libros/kubernetes-101/erratas>

Arquitectura de Kubernetes

La arquitectura de Kubernetes permite orquestar y ejecutar contenedores a escala de una manera distribuida.

Para ello emplea un conjunto de componentes que interactúan entre sí a través de una API para mantener el estado definido por los usuarios.

Estos componentes se agrupan en dos grandes bloques:

- El control plane, que toma las decisiones globales sobre el clúster: qué Pods se ejecutan, dónde se ejecutan y cómo reaccionar cuando algo cambia respecto al estado deseado.
- Los nodos, las máquinas (físicas o virtuales) donde realmente se ejecutan los Pods.

En clústeres desplegados con herramientas como `kubeadm`²⁵, los propios componentes del control plane se ejecutan como Pods sobre los nodos de control plane. Por eso, estos nodos también necesitan ejecutar `kubelet`²⁶, aunque no alojen cargas de trabajo de usuario.

De todos ellos, dos hay que tener en la cabeza desde la primera página, porque el resto del libro los da por sabidos: el `kube-apiserver`, por donde entra absolutamente todo, y el `kubelet`, que es quien acaba ejecutando. Son los dos que vienen a continuación.

Los demás se explican donde hacen falta. `kube-proxy` abre el capítulo de Red, porque no se entiende sin el Service. El `kube-scheduler` abre el de Scheduling, que es donde se decide dónde cae cada Pod. Y la admisión vive en el de Seguridad, pegada a las políticas que la usan. Lo que queda —`etcd`, el inventario de controladores, la interfaz con el runtime de contenedores y los Leases— está en el Apéndice D, que puedes leer hoy o el día que lo necesites.

²⁵`kubeadm`. Herramienta oficial para instalar y arrancar un clúster de Kubernetes conforme a las buenas prácticas. Despliega los componentes del control plane como Pods sobre los nodos de control.

²⁶`kubelet`. Agente que se ejecuta en cada nodo. Recibe del API Server los PodSpec que le corresponden, pide al runtime que arranque los contenedores y reporta el estado del nodo y de los Pods.

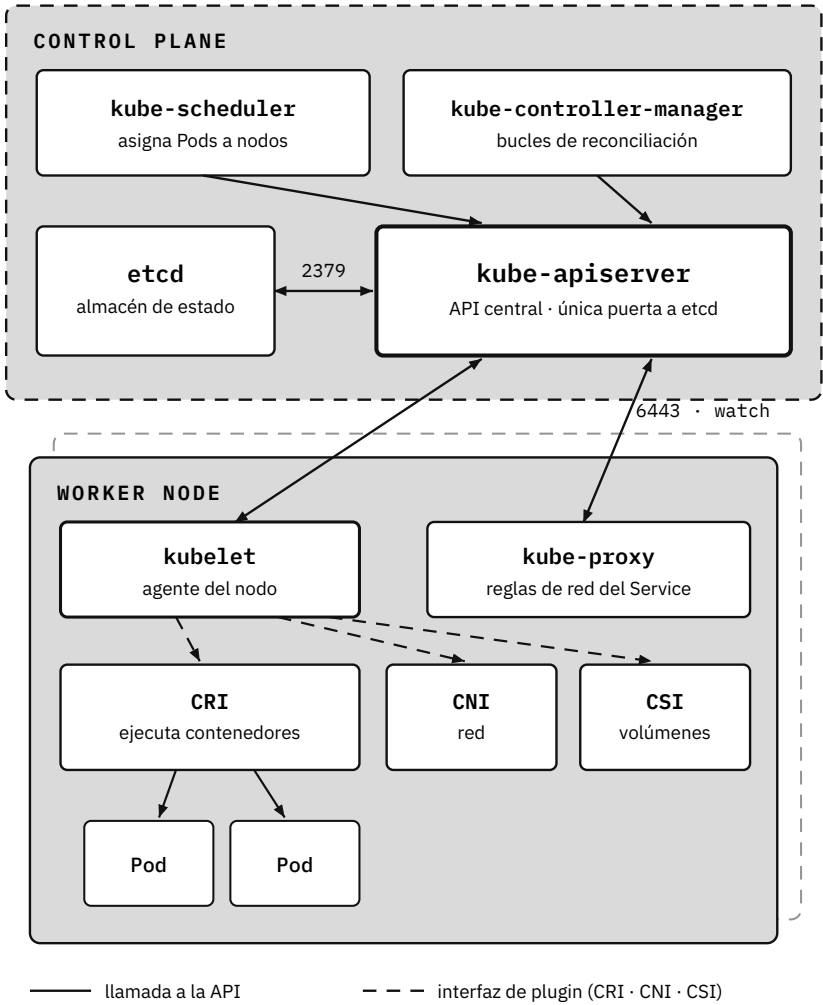


Figura 3: Arquitectura del clúster: control plane y nodos

Fíjate en que todas las flechas pasan por el kube-apiserver: ningún componente habla directamente con otro, ni siquiera los que viven en la misma máquina. Eso es lo que hace que solo haya un punto que proteger de verdad, y también que solo haya uno que se pueda saturar.

Componentes clave

kube-apiserver

El kube-apiserver es el punto por el que entran todas las peticiones del clúster, tanto las del resto de componentes como las de sus usuarios, ya sea para crear objetos nuevos o para consultar su estado. Se interactúa con él mediante una API REST, y es también quien almacena y actualiza esa información en etcd.

Antes de ejecutar una petición, esta es autenticada, autorizada y analizada por los admission controllers. En las siguientes páginas describiremos cada uno de estos elementos.

Por defecto, el puerto utilizado por la API es el 6443 y está protegido por TLS¹.

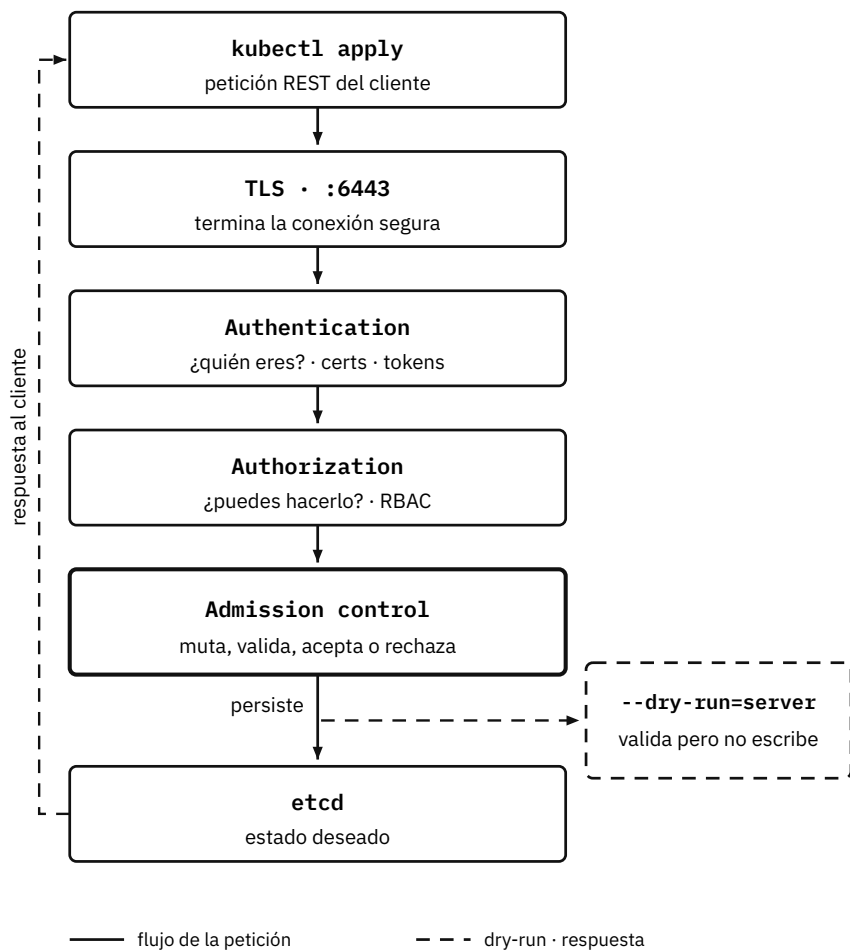


Figura 4: El ciclo de una petición en el kube-apiserver

Saber qué etapa te ha rechazado ahorra medio diagnóstico: un Unauthorized viene de la autenticación, un Forbidden de la autorización y casi cualquier otro mensaje raro, de la admisión. --dry-run recorre todo el pipeline, admisión incluida, pero no se guarda en etcd.

Es posible configurar más de un método de autenticación. Al menos deben existir dos métodos:

- Uno para autenticar las peticiones que provienen de las Service Accounts (SA)
- Otro para autenticar a los usuarios del clúster

En el caso de que haya más de un módulo de autenticación activado, se van ejecutando secuencialmente hasta que uno de ellos puede validar correctamente la petición.

Los módulos de autenticación y autorización se explican en el capítulo de Seguridad.

kubelet

kubelet se ejecuta como una aplicación en cada uno de los nodos que componen el clúster, incluidos los nodos de control plane en clústeres autogestionados.

Sus responsabilidades principales son:

- Registrar el nodo en el control plane.
- Obtener las especificaciones de los Pods que debe ejecutar.
- Crear, modificar y borrar los contenedores asociados a un Pod, delegando la ejecución real en el container runtime a través de la Container Runtime Interface (CRI).
- Manejar las diferentes probes (liveness, readiness y startup) configuradas para cada uno de los contenedores.
- Informar al control plane sobre el estado de los Pods que se están ejecutando en el nodo.

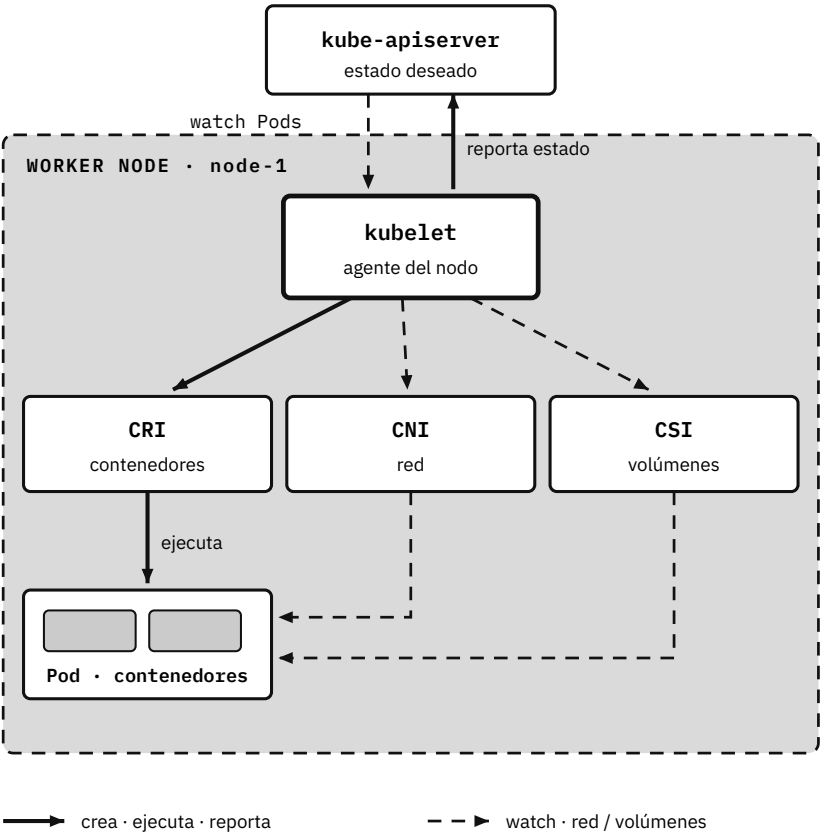


Figura 5: El kubelet y los Pods de su nodo

El kubelet no espera órdenes: vigila el API server y actúa sobre lo que le toca. Por eso un nodo que pierde la conexión con el control plane sigue ejecutando sus Pods tan tranquilo; lo que deja de haber no son contenedores, son cambios.

Comunicación entre componentes

Kubernetes implementa el patrón hub-and-spoke²⁷, en el que el kube-apiserver actúa como único punto central de comunicación. Salvo el propio API Server y la excepción del kubelet (que verás a continuación), ningún componente expone puertos para recibir conexiones remotas de forma directa.

Acceso al API Server

Se accede principalmente mediante HTTPS. Desde dentro del clúster se hace a través del Service kubernetes, que escucha en el puerto 443 y redirige al 6443 del API server; desde fuera, directamente al 6443. Soporta múltiples estrategias de autenticación: certificados X.509, tokens de ServiceAccount, OpenID Connect, webhook, etc.

Autenticación de nodos y Pods

Los nodos se autentican con certificados de cliente (generados normalmente por kubeadm o herramientas similares).

Los Pods utilizan ServiceAccounts. Kubernetes monta automáticamente el token y el certificado CA en cada Pod, permitiéndole comunicarse de forma segura con el API Server.

Comunicación entre el API Server y kubelet

Esta comunicación es bidireccional, y por ella pasan varias de las operaciones que lanzarás con kubectl, la herramienta de línea de comandos del capítulo siguiente:

- Obtener logs de los Pods (kubectl logs).
- kubectl port-forward.
- kubectl attach / kubectl exec.
- Recopilación de métricas y estado del nodo.

Se establece sobre el puerto HTTPS del kubelet (10250), el único puerto que un componente distinto del API Server expone para recibir conexiones remotas

²⁷Hub-and-spoke: Topología de comunicación en la que todas las piezas hablan con un único punto central (el hub) y nunca entre sí (los spokes). Reduce el número de conexiones que hay que asegurar y deja un solo sitio donde autenticar, autorizar y auditar; a cambio, ese centro es crítico y conviene desplegarlo en alta disponibilidad. En Kubernetes el hub es el kube-apiserver.

directas. Por defecto, el API Server no valida el certificado del kubelet (parámetro `--kubelet-certificate-authority` vacío), lo que representa un riesgo de seguridad que debe mitigarse en producción.

Otras conexiones del API Server

El API Server también puede iniciar conexiones hacia:

- Nodos (a través del kubelet).
- Pods (usando el kubelet como proxy).
- Services (para proxying interno).

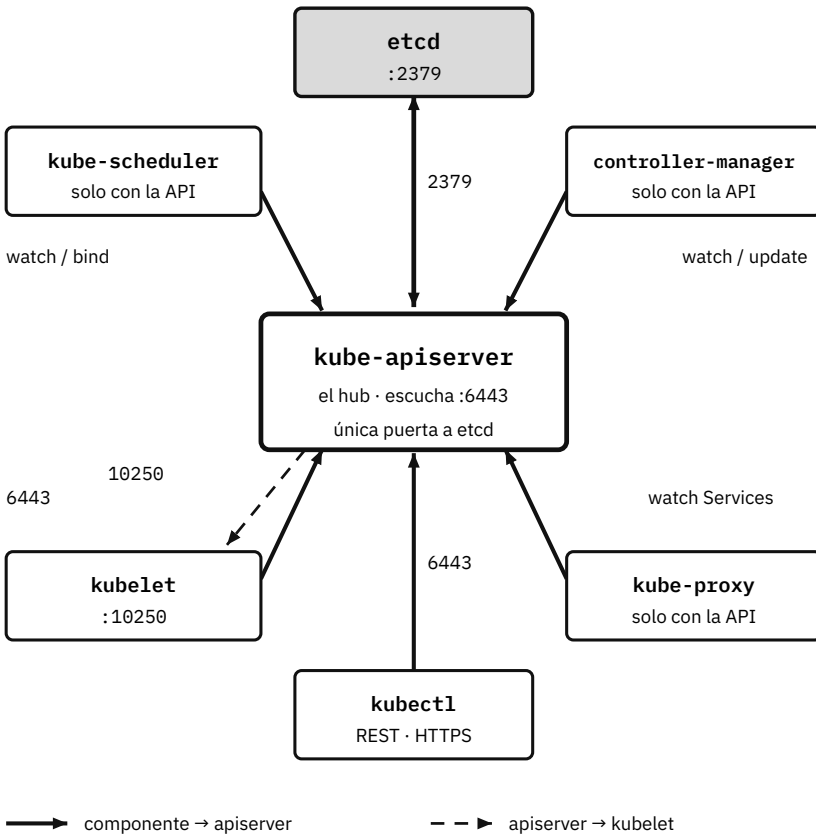


Figura 6: Comunicación entre los componentes del clúster

Ningún componente habla con otro directamente y solo el API Server se comunica con el etcd. Por lo que solo se necesitan dos puertos: 6443 hacia el control plane y 10250 hacia los kubelets.

Tu clúster probablemente no se parece a este

Hay dos formas de tener un clúster. En una lo montas y lo mantienes tú, con `kubeadm` o con lo que prefieras, y entonces los componentes del control plane son Pods que puedes listar, cuyos logs puedes leer y cuyos parámetros puedes cambiar. En la otra alguien te vende el control plane ya montado, como EKS, AKS o GKE. Conviene saber qué cambia entre las dos opciones.

En un clúster gestionado, `kubectl get pods -n kube-system` no devuelve el `kube-apiserver`, ni el `kube-scheduler`, ni `etcd`: corren en una infraestructura interna que no es tuya y que ni siquiera aparece en `kubectl get nodes`. Verás CoreDNS, el `kube-proxy` y los `DaemonSets` del proveedor, y poco más. No es que estén rotos: están en otro sitio.

Ahí está también el reparto de responsabilidades. Como no tocas los parámetros del `kube-apiserver`, hay feature gates que sencillamente no vas a poder activar, ni opciones que ajustar cuando algo va justo. A cambio, el backup de `etcd`, las actualizaciones del control plane y su alta disponibilidad dejan de ser cosa tuya, y ese suele ser el motivo real por el que se contrata: menos tiempo de mantenimiento a cambio de menos control.

La buena noticia es la que hace que este capítulo siga valiendo: no cambia ni un objeto. El Pod, el Deployment y el Service que vas a ver a partir de aquí son idénticos en un portátil, en un clúster de tres máquinas y en EKS. Lo que cambia es a quién llamas cuando dejan de funcionar.

Quién implementa qué

Queda una cosa por decir de este capítulo, y es la que más ahorra al leer el resto del libro: **Kubernetes define objetos y delega el trabajo**. Casi nada de lo que vas a declarar en estas páginas lo ejecuta Kubernetes; lo ejecuta un componente que alguien instaló, y que se puede cambiar por otro.

Lo que declaras	Quién lo hace de verdad	Qué pasa si no está
Que los Pods tengan IP y se hablen	El plugin de red, CNI (Calico, Cilium, Flannel...)	Los Pods se quedan en <code>ContainerCreating</code> para siempre
Que un Pod se ejecute	El runtime, por CRI (containerd, CRI-O)	El nodo ni siquiera llega a <code>Ready</code>

Lo que declaras	Quién lo hace de verdad	Qué pasa si no está
Que un PVC se convierta en disco	El driver de almacenamiento, CSI	El PVC se queda en Pending sin decir por qué
Que una NetworkPolicy bloquee tráfico	El mismo CNI, si la admite	Se aplica sin error y no bloquea nada
Que un Ingress publique tu web	Un ingress controller que instalas tú	El objeto existe, y no responde nadie
Un Gateway o un HTTPRoute	Las CRDs de Gateway API y su controlador	kubectl get gateway responde que ese tipo no existe
Que existan métricas de uso de CPU y memoria	El metrics-server	El HPA no escala y no hay datos de consumo
Que una GPU se reparta	El driver del fabricante	El Pod se queda en Pending

La columna de la derecha es la razón de la tabla. Todos esos fallos comparten una característica que los hace difíciles la primera vez: **Kubernetes acepta el manifiesto igualmente**. No hay error de sintaxis ni rechazo en la admisión, porque el objeto es perfectamente válido; lo que falta es alguien que lo atienda. Un clúster sin ingress controller acepta tus Ingress con la misma naturalidad que uno que sí lo tiene.

Que sea así tiene un motivo: es lo que permite que el mismo YAML funcione en un portátil y en tres nubes distintas. Y tiene un precio, que es este capítulo entero: **saber quién hace cada cosa es lo que convierte un “no funciona” en una pregunta con respuesta**.

Laboratorio. Módulo *El clúster por dentro*: un recorrido por un clúster kubeadm componente a componente, con lo que aquí se cuenta y lo que amplía el Apéndice D.

Apéndice D: El clúster por dentro

El capítulo de Arquitectura se queda con lo que hay que tener en la cabeza para leer el resto del libro: quién recibe las peticiones y quién ejecuta los contenedores. Aquí está lo que hay por debajo.

No hace falta para desplegar una aplicación, y por eso no está en el cuerpo del libro. Hace falta el día que un clúster va lento y nadie sabe por qué, cuando toca dimensionar el control plane, o cuando hay que actualizar de versión y conviene saber qué se mueve por dentro.

etcd

En todo momento Kubernetes monitoriza, analiza y realiza las operaciones necesarias para poder mantener el estado deseado por los usuarios.

Para almacenar esta información hace uso del almacén de clave-valor distribuido `etcd`.

Al estar distribuido en varias instancias, `etcd` ofrece alta disponibilidad: si una instancia falla, el resto puede seguir respondiendo peticiones sin interrupción. Esto es distinto de recuperarse de un fallo catastrófico de todo el clúster, que depende de disponer de backups de `etcd` (lo vemos un poco más abajo).

Las principales características de `etcd`:

- **Consistencia:** utiliza el algoritmo Raft³⁹, garantizando que los nodos tengan la misma versión de los datos en todo momento.
- **Alta disponibilidad:** los datos se pueden replicar en múltiples instancias, lo que permite seguir accediendo a los datos en caso de que una instancia falle.
- **Interfaz simple:** desde la versión 3, `etcd` expone su API de forma nativa mediante gRPC⁴⁰, con operaciones CRUD (create, read, update, delete). También

³⁹Raft. Algoritmo de consenso distribuido que utiliza `etcd` para elegir un líder y garantizar que todas las réplicas mantienen los mismos datos. Explicación interactiva: <https://raft.github.io/>

⁴⁰gRPC. Protocolo de llamada a procedimiento remoto (RPC) sobre HTTP/2 que serializa los mensajes en formato binario (Protocol Buffers). Es más eficiente que REST/JSON y admite streaming bidireccional.

existe un gateway que permite acceder a esta misma API a través de HTTP/JSON.

- **Watch API:** permite suscribirse a cambios en claves específicas para obtener notificaciones cuando se produce un cambio en los valores de la clave.

etcd es una parte crítica de Kubernetes debido a algunas de sus limitaciones que conviene tener en cuenta:

- **Latencia:** una alta latencia puede provocar timeouts y cambios frecuentes en el líder (leader).
- **Acceso a disco:** es recomendable la utilización de discos SSD. El rendimiento de etcd está muy relacionado con el rendimiento del disco utilizado.
- **Limitaciones de tamaño:** el tamaño aproximado de un único valor (value) es de ~1 MiB. Por defecto, el tamaño máximo del almacén es de 2 GB, ampliable hasta un máximo recomendado de 8 GB. El tiempo de arranque es proporcional al tamaño de los datos. A mayor tamaño, mayor tiempo de arranque.

Para finalizar, y en caso de que no utilices un clúster de un cloud provider, debes aplicar las siguientes prácticas:

Backups: realizar backups frecuentes permite restaurar el estado del clúster en caso de tener un fallo catastrófico. `etcdctl` puede ayudarte en esta tarea.

Monitorización: tener configurada una correcta monitorización del uso de CPU, memoria y espacio en disco te ayuda a detectar de manera proactiva un posible problema.

Alta disponibilidad: utilizar varias instancias para ejecutar el servicio evita tener un punto único de fallo.

etcd es un proyecto que forma parte de la Cloud Native Computing Foundation (CNCF - <https://www.cncf.io/>), puedes leer más sobre él en su página web <https://etcd.io/>.

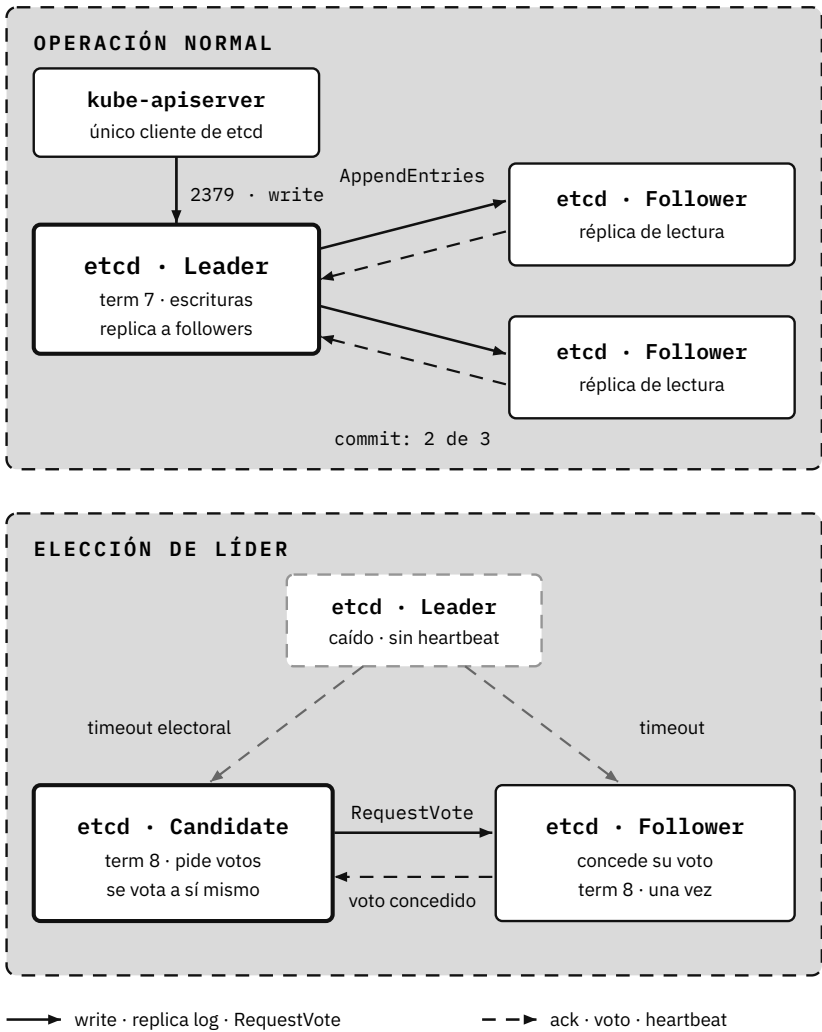


Figura 53: etcd: elección de líder con Raft

Una escritura se confirma cuando la acepta la mayoría, no todos: en un clúster de tres, con dos basta. Por eso etcd se despliega siempre en número impar, y por eso perder dos de tres miembros deja el clúster en solo lectura.

kube-controller-manager

El kube-controller-manager es un único proceso que ejecuta internamente múltiples controladores (también llamados control loops), cada uno responsable de monitorizar un tipo de recurso y de asegurarse de que se encuentra en el estado correcto.

A continuación, algunos de los controladores que corren dentro del kube-controller-manager. Los recursos que gestionan los has ido viendo capítulo a capítulo; lo que enseña esta lista es que no hay un proceso por cada uno: son bucles dentro del mismo binario.

- **Certificate Controller:** gestiona automáticamente la emisión y renovación de certificados.
- **CronJob Controller:** ejecuta jobs según una programación temporal definida.
- **DaemonSet Controller:** asegura que todos los nodos (o algunos filtrados) ejecuten una copia de un Pod específico.
- **Deployment Controller:** gestiona el despliegue declarativo de aplicaciones y sus actualizaciones progresivas.
- **EndpointSlice Controller:** crea y actualiza objetos EndpointSlice que conectan Services con Pods.
- **HorizontalPodAutoscaling Controller:** escala automáticamente el número de Pods basándose en métricas definidas.
- **Job Controller:** supervisa y gestiona tareas de un solo uso hasta su finalización.
- **Namespace Controller:** gestiona el ciclo de vida de los Namespaces y su limpieza.
- **Node Controller:** monitoriza los nodos y responde cuando no están disponibles.
- **ReplicaSet Controller:** mantiene en ejecución el número de réplicas declarado por cada ReplicaSet.
- **ResourceQuota Controller:** aplica las restricciones de recursos por Namespace.
- **Service Account Controller:** crea cuentas de servicio predeterminadas en nuevos Namespaces.
- **StatefulSet Controller:** gestiona el despliegue y escalado de aplicaciones con estado.

cloud-controller-manager

Cuando el clúster se ejecuta sobre un proveedor cloud (AWS, GCP, Azure...), existe un componente adicional llamado `cloud-controller-manager`. Su función es aislar la lógica específica de cada proveedor del resto del control plane, de forma que el `kube-controller-manager` pueda seguir siendo independiente del proveedor sobre el que se ejecuta.

Entre sus responsabilidades habituales se encuentran:

- **Node Controller:** comprueba en el proveedor cloud si un nodo ha sido eliminado tras dejar de responder.
- **Route Controller:** configura las rutas de red necesarias en la infraestructura del proveedor.
- **Service Controller:** crea, actualiza y elimina los load balancers del proveedor cuando se crean Services de tipo LoadBalancer.

Cómo se ejecutan los contenedores: CRI, runtimes y OCI

Cada nodo necesita un container runtime: el software responsable de descargar imágenes, crearlas y ejecutar contenedores. Kubernetes no implementa esta lógica directamente, sino que se comunica con el runtime a través de la Container Runtime Interface (CRI), una API estándar que permite usar cualquier implementación de runtime sin modificar `kubelet`. Los más utilizados actualmente son `containerd` y CRI-O.

No siempre fue así. En sus primeras versiones, Kubernetes integraba Docker de forma totalmente acoplada: el `kubelet` llamaba directamente a la API de Docker, sin ninguna capa de abstracción de por medio. Si querías utilizar otro runtime, no había forma de hacerlo. CRI apareció en la versión 1.5 precisamente para romper ese acoplamiento. Como Docker no hablaba CRI de forma nativa, se añadió `dockershim`, una capa de traducción dentro del propio `kubelet` que permitía seguir usando Docker mientras el ecosistema migraba hacia runtimes que sí la implementaban de forma nativa, como `containerd` o CRI-O. Finalmente, en la versión 1.24 (2022), `dockershim` se eliminó del `kubelet`; Docker dejó de soportarse “de fábrica”, aunque sigue siendo posible usarlo a través de `cri-dockerd`, un adaptador mantenido por la comunidad.

CRI resuelve el problema de cómo se comunican el kubelet y el runtime que ejecuta los contenedores, pero ¿cómo se resuelve el problema de ejecutar contenedores con distintos niveles de aislamiento dentro del mismo nodo? Aquí es donde entra RuntimeClass. En la práctica, un nodo ejecuta una única implementación CRI (por ejemplo, containerd), no varias a la vez. Lo que RuntimeClass permite seleccionar, por Pod, es qué runtime de bajo nivel (handler OCI) debe usar esa implementación CRI para ejecutar los contenedores de dicho Pod: por ejemplo runc (el estándar, basado en namespaces y cgroups de Linux), Kata Containers (aislamiento mediante una VM ligera) o gVisor (un kernel de aplicación en espacio de usuario que intercepta y reimplementa las syscalls). Esto resulta útil cuando se necesita ejecutar cargas de trabajo no confiables con mayor aislamiento, sin tener que dedicarles un nodo o clúster aparte.

Dos retiradas anunciadas que conviene tener en el radar. Esta capa lleva años siendo invisible y se hace notar justo al actualizar. La primera: la retirada de los flags de configuración del kubelet, que estaba prevista para la 1.37, se ha aplazado a la **1.38** precisamente para que coincida con el fin de soporte de containerd v1.7; desde la 1.37, kubeadm ya avisa en su preflight si el runtime instalado se queda corto. La segunda: cgroup v1, deprecado desde la 1.35, sigue funcionando sin cambios en la 1.37, pero su eliminación está planificada (KEP-5573). Las dos se comprueban sobre los nodos, no sobre el control plane, y las dos son de las que conviene mirar antes de subir de versión y no después.

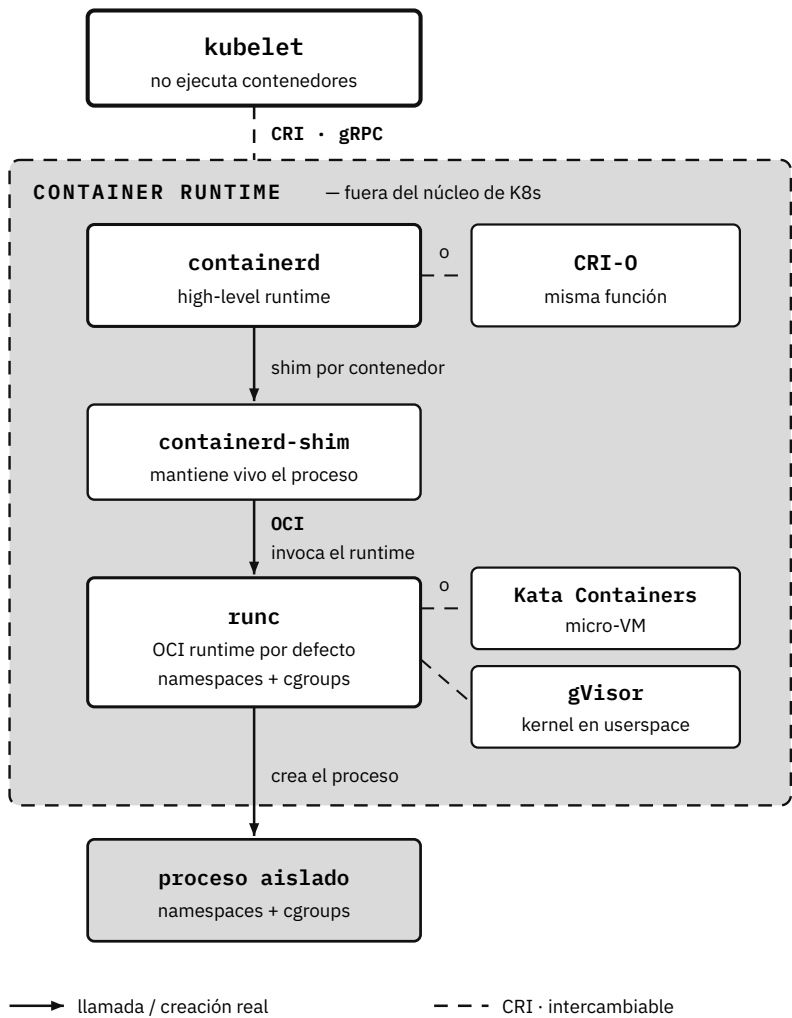


Figura 54: Del kubelet al contenedor: CRI, runtimes y OCI

Las dos interfaces existen para que las piezas sean intercambiables: puedes sustituir containerd por CRI-O, o runc por gVisor o Kata, sin que Kubernetes se entere.

Leases

Ya están todas las piezas sobre la mesa, y con ellas aparecen dos preguntas que ningún componente por separado puede responder: si un nodo lleva un rato callado, ¿está caído o solo lento? Y si el `kube-scheduler` corre en tres réplicas para no tener un punto único de fallo, ¿cuál de las tres manda?

Kubernetes resuelve las dos con el mismo objeto, y es de los más pequeños de la API. Un Lease representa un **testigo con fecha de caducidad**: un componente declara que lo posee durante unos segundos y debe renovarlo periódicamente. Si deja de renovarlo, se asume que ha fallado.

- **¿Quién está vivo?** En un sistema distribuido no se puede distinguir un componente caído de uno lento; la única respuesta práctica es exigir señales periódicas. Los `kubelets` renuevan su Lease (`kube-node-lease`) cada pocos segundos y, si dejan de hacerlo, el nodo pasa a `NotReady`.
- **¿Quién está al mando?** Cuando se ejecutan varias réplicas de un componente para tener alta disponibilidad, solo una debe estar activa. El Lease actúa como un lock distribuido: quien consigue renovarlo es el líder, y si falla, otra réplica toma el relevo.

Por qué importan:

- *Escalabilidad*: antes los `kubelets` reportaban su salud actualizando el objeto `Node` completo, una escritura grande y frecuente contra `etcd`. Sustituirla por un Lease redujo drásticamente ese tráfico y es lo que hace viables los clústeres de miles de nodos.
- *Alta disponibilidad*: sin leader election, dos schedulers programarían el mismo Pod. Los Leases dan ese lock reutilizando `etcd`, sin infraestructura adicional.
- *Simplicidad*: un único primitivo cubre detección de fallos, elección de líder y descubrimiento de instancias.

Ejemplos de uso en el control plane:

- `kube-apiserver` publica su presencia mediante Leases.
- `kube-controller-manager` y `kube-scheduler` los usan para leader election (solo uno es activo aunque se ejecuten en HA).
- Los `kubelets` crean y renuevan su Lease en `kube-node-lease` para informar de su estado.

Ese kube-node-lease es uno de los cuatro Namespaces que Kubernetes trae de fábrica, y lo verás enseguida en el capítulo “Objetos de Kubernetes”. Si alguna vez quieres mirar los testigos por dentro: `kubectl get lease -n kube-node-lease`.

Laboratorio. Módulo *El clúster por dentro*, el que cierra la parte práctica: ahí ves en marcha lo que este apéndice cuenta.

HASTA AQUÍ EL REGALO

Quedan catorce capítulos

Acabas de leer el arranque de Kubernetes 101: la guía en español para entender Kubernetes desde cero.

- **15 capítulos**

De qué es un contenedor a observabilidad.

- **5 apéndices**

Glosario, el sistema completo, desde cuándo funciona cada pieza, el clúster por dentro y una chuleta de kubectl.

- **55 laboratorios**

Un clúster real en tu navegador. Sin instalar nada.

- **7 challenges**

Te entregan un clúster roto. Tú lo arreglas.

Disponible el 1 de septiembre

Kindle 12,99 EUR · Papel 24,99 EUR

Reservar en Amazon

www.javivela.dev/libros/kubernetes-101

© 2026 Javier Vela Aylón · Extracto de Kubernetes 101 · Uso personal